

CS 4782 Homework 1

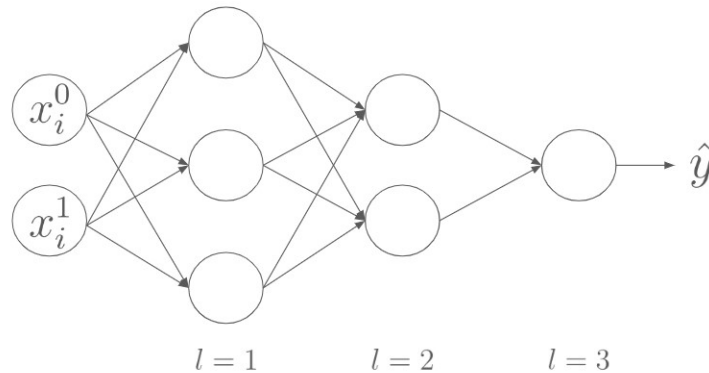
Due: 2/12/26 11:59 PM on Gradescope

Late submissions excepted until 2/14/26 11:59 PM

Note: For homework, you can work in teams of up to 2 people. Please include your teammates' NetIDs and names on the front page and form a group on Gradescope. (Please answer all questions within each paragraph.) Please show all the relevant steps in your solutions.

Problem 1: Backpropagation

1(a). Suppose you have the following 3-layer fully connected neural network for a regression problem ($y \in \mathbb{R}^+$).



We want to run empirical risk minimization on this neural network using the MSE loss function, which for a single example (x, y) is given by

$$L(\hat{y}, y) = (y - \hat{y})^2.$$

The following equations represent the forward pass through this neural network. Recall the weights and bias of the l^{th} layer can be written as $W^{[l]}$ and $b^{[l]}$ respectively. Assume that the activation function used, $\sigma(x)$, is $\text{ReLU}(x) = \max(0, x)$.

$$\begin{aligned} z^{[0]} &= x \\ a^{[1]} &= W^{[1]}z^{[0]} + b^{[1]} \\ z^{[1]} &= \text{ReLU}(a^{[1]}) \\ a^{[2]} &= W^{[2]}z^{[1]} + b^{[2]} \\ z^{[2]} &= \text{ReLU}(a^{[2]}) \\ a^{[3]} &= W^{[3]}z^{[2]} + b^{[3]} \\ z^{[3]} &= \text{ReLU}(a^{[3]}) \\ \hat{y} &= z^{[3]} \end{aligned} \tag{1}$$

Suppose we have the following training data point

$$\left(\begin{bmatrix} 2 \\ 1 \end{bmatrix}, 10 \right)$$

and the weights and bias terms of our neural network are

$$W^{[1]} = \begin{bmatrix} 1 & 0 \\ 3 & 4 \\ 1 & 2 \end{bmatrix} \quad b^{[1]} = \begin{bmatrix} 0 \\ -2 \\ 5 \end{bmatrix} \quad W^{[2]} = \begin{bmatrix} 2 & 6 & 1 \\ 0 & 3 & 1 \end{bmatrix} \quad b^{[2]} = \begin{bmatrix} -23 \\ -11 \end{bmatrix} \quad W^{[3]} = \begin{bmatrix} -2 & 3 \end{bmatrix} \quad b^{[3]} = 18$$

Calculate the values of each of the terms listed in equation (1) above.

$$z^{[0]} = x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$a^{[1]} = W^{[1]}z^{[0]} + b^{[1]} = \begin{bmatrix} 1 & 0 \\ 3 & 4 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ -2 \\ 5 \end{bmatrix} = \begin{bmatrix} 2 \\ 8 \\ 9 \end{bmatrix}$$

$$z^{[1]} = \text{ReLU}(a^{[1]}) = \begin{bmatrix} 2 \\ 8 \\ 9 \end{bmatrix}$$

$$a^{[2]} = W^{[2]}z^{[1]} + b^{[2]} = \begin{bmatrix} 2 & 6 & 1 \\ 0 & 3 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 8 \\ 9 \end{bmatrix} + \begin{bmatrix} -23 \\ -11 \end{bmatrix} = \begin{bmatrix} 38 \\ 22 \end{bmatrix}$$

$$z^{[2]} = \text{ReLU}(a^{[2]}) = \begin{bmatrix} 38 \\ 22 \end{bmatrix}$$

$$a^{[3]} = W^{[3]}z^{[2]} + b^{[3]} = \begin{bmatrix} -2 & 3 \end{bmatrix} \begin{bmatrix} 38 \\ 22 \end{bmatrix} + 18 = 8$$

$$\hat{y} = z^{[3]} = \text{ReLU}(a^{[3]}) = 8$$

1(b). Now using the backpropagation algorithm presented in Lecture 2, Slide 67, derive the series of terms backpropagation would compute to find gradients for this neural network, i.e. complete the following equations. You

may use $\text{ReLU}'(x)$ to denote the gradient of the ReLU function.

$$\begin{aligned}\frac{\partial L}{\partial \hat{y}} &= \\ \frac{\partial L}{\partial z^{[3]}} &= \\ \frac{\partial L}{\partial a^{[3]}} &= \end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial W^{[3]}} &= \\ \frac{\partial L}{\partial b^{[3]}} &= \\ \frac{\partial L}{\partial z^{[2]}} &= \\ \frac{\partial L}{\partial a^{[2]}} &= \end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial W^{[2]}} &= \\ \frac{\partial L}{\partial b^{[2]}} &= \\ \frac{\partial L}{\partial z^{[1]}} &= \\ \frac{\partial L}{\partial a^{[1]}} &= \end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial W^{[1]}} &= \\ \frac{\partial L}{\partial b^{[1]}} &= \\ \frac{\partial L}{\partial z^{[0]}} &= \\ \frac{\partial L}{\partial x} &= \end{aligned}$$

$$\begin{aligned}
\frac{\partial L}{\partial \hat{y}} &= \frac{\partial}{\partial \hat{y}} (y - \hat{y})^2 = -2(y - \hat{y}) \\
\frac{\partial L}{\partial z^{[3]}} &= \frac{\partial L}{\partial \hat{y}} = -2(y - \hat{y}) \\
\frac{\partial L}{\partial a^{[3]}} &= \frac{\partial L}{\partial z^{[3]}} \frac{\partial z^{[3]}}{\partial a^{[3]}} = \frac{\partial L}{\partial z^{[3]}} \left(\frac{\partial}{\partial a^{[3]}} \text{ReLU}(a^{[3]}) \right) = -2(y - \hat{y}) \odot \text{ReLU}'(a^{[3]})
\end{aligned}$$

$$\begin{aligned}
\frac{\partial L}{\partial W^{[3]}} &= \frac{\partial L}{\partial a^{[3]}} \frac{\partial a^{[3]}}{\partial W^{[3]}} = \frac{\partial L}{\partial a^{[3]}} \left(\frac{\partial}{\partial W^{[3]}} W^{[3]} z^{[2]} + b^{[3]} \right) = \frac{\partial L}{\partial a^{[3]}} (z^{[2]})^T \\
\frac{\partial L}{\partial b^{[3]}} &= \frac{\partial L}{\partial a^{[3]}} \frac{\partial a^{[3]}}{\partial b^{[3]}} = \frac{\partial L}{\partial a^{[3]}} \left(\frac{\partial}{\partial b^{[3]}} W^{[3]} z^{[2]} + b^{[3]} \right) = \frac{\partial L}{\partial a^{[3]}} \\
\frac{\partial L}{\partial z^{[2]}} &= \frac{\partial L}{\partial a^{[3]}} \frac{\partial a^{[3]}}{\partial z^{[2]}} = \frac{\partial L}{\partial a^{[3]}} \left(\frac{\partial}{\partial z^{[2]}} W^{[3]} z^{[2]} + b^{[3]} \right) = (W^{[3]})^T \frac{\partial L}{\partial a^{[3]}} \\
\frac{\partial L}{\partial a^{[2]}} &= \frac{\partial L}{\partial z^{[2]}} \frac{\partial z^{[2]}}{\partial a^{[2]}} = \frac{\partial L}{\partial z^{[2]}} \left(\frac{\partial}{\partial a^{[2]}} \text{ReLU}(a^{[2]}) \right) = \left((W^{[3]})^T \frac{\partial L}{\partial a^{[3]}} \right) \odot \text{ReLU}'(a^{[2]})
\end{aligned}$$

$$\begin{aligned}
\frac{\partial L}{\partial W^{[2]}} &= \frac{\partial L}{\partial a^{[2]}} (z^{[1]})^T \\
\frac{\partial L}{\partial b^{[2]}} &= \frac{\partial L}{\partial a^{[2]}} \\
\frac{\partial L}{\partial z^{[1]}} &= (W^{[2]})^T \frac{\partial L}{\partial a^{[2]}} \\
\frac{\partial L}{\partial a^{[1]}} &= \left((W^{[2]})^T \frac{\partial L}{\partial a^{[2]}} \right) \odot \text{ReLU}'(a^{[1]})
\end{aligned}$$

$$\begin{aligned}
\frac{\partial L}{\partial W^{[1]}} &= \frac{\partial L}{\partial a^{[1]}} (z^{[0]})^T \\
\frac{\partial L}{\partial b^{[1]}} &= \frac{\partial L}{\partial a^{[1]}} \\
\frac{\partial L}{\partial z^{[0]}} &= (W^{[1]})^T \frac{\partial L}{\partial a^{[1]}} \\
\frac{\partial L}{\partial x} &= \frac{\partial L}{\partial z^{[0]}} = (W^{[1]})^T \frac{\partial L}{\partial a^{[1]}}
\end{aligned}$$

1(c). Given the same training point, weights, and bias terms as in part (a), calculate the gradients of each of the weights and bias terms.

$$\begin{aligned}\frac{\partial L}{\partial \hat{y}} &= \frac{\partial L}{\partial z^{[3]}} = -2(y - \hat{y}) = -4 \\ \frac{\partial L}{\partial a^{[3]}} &= -2(y - \hat{y}) \odot \text{ReLU}'(a^{[3]}) = -4 * \text{ReLU}'(8) = -4\end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial W^{[3]}} &= \frac{\partial L}{\partial a^{[3]}} (z^{[2]})^T = -4 \begin{bmatrix} 38 \\ 22 \end{bmatrix}^T = \begin{bmatrix} -152 & -88 \end{bmatrix} \\ \frac{\partial L}{\partial b^{[3]}} &= \frac{\partial L}{\partial a^{[3]}} = -4 \\ \frac{\partial L}{\partial z^{[2]}} &= (W^{[3]})^T \frac{\partial L}{\partial a^{[3]}} = \begin{bmatrix} -2 & 3 \end{bmatrix}^T - 4 = \begin{bmatrix} 8 \\ -12 \end{bmatrix} \\ \frac{\partial L}{\partial a^{[2]}} &= \left((W^{[3]})^T \frac{\partial L}{\partial a^{[3]}} \right) \odot \text{ReLU}'(a^{[2]}) = \begin{bmatrix} 8 \\ -12 \end{bmatrix} \odot \text{ReLU}' \left(\begin{bmatrix} 38 \\ 22 \end{bmatrix} \right) = \begin{bmatrix} 8 \\ -12 \end{bmatrix}\end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial W^{[2]}} &= \frac{\partial L}{\partial a^{[2]}} (z^{[1]})^T = \begin{bmatrix} 8 \\ -12 \end{bmatrix} \begin{bmatrix} 2 \\ 8 \\ 9 \end{bmatrix}^T = \begin{bmatrix} 16 & 64 & 72 \\ -24 & -96 & -108 \end{bmatrix} \\ \frac{\partial L}{\partial b^{[2]}} &= \frac{\partial L}{\partial a^{[2]}} = \begin{bmatrix} 8 \\ -12 \end{bmatrix} \\ \frac{\partial L}{\partial z^{[1]}} &= (W^{[2]})^T \frac{\partial L}{\partial a^{[2]}} = \begin{bmatrix} 2 & 6 & 1 \\ 0 & 3 & 1 \end{bmatrix}^T \begin{bmatrix} 8 \\ -12 \end{bmatrix} = \begin{bmatrix} 16 \\ 12 \\ -4 \end{bmatrix} \\ \frac{\partial L}{\partial a^{[1]}} &= \left((W^{[2]})^T \frac{\partial L}{\partial a^{[2]}} \right) \odot \text{ReLU}'(a^{[1]}) = \begin{bmatrix} 16 \\ 12 \\ -4 \end{bmatrix} \odot \text{ReLU}' \left(\begin{bmatrix} 2 \\ 8 \\ 9 \end{bmatrix} \right) = \begin{bmatrix} 16 \\ 12 \\ -4 \end{bmatrix}\end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial W^{[1]}} &= \frac{\partial L}{\partial a^{[1]}} (z^{[0]})^T = \begin{bmatrix} 16 \\ 12 \\ -4 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix}^T = \begin{bmatrix} 32 & 16 \\ 24 & 12 \\ -8 & -4 \end{bmatrix} \\ \frac{\partial L}{\partial b^{[1]}} &= \frac{\partial L}{\partial a^{[1]}} = \begin{bmatrix} 16 \\ 12 \\ -4 \end{bmatrix}\end{aligned}$$

Problem 2: Efficiency of Random Hyperparameter Search (10 points)

Hyperparameter search is an effective but expansive process in machine learning especially for small models. Let's say we are training a relatively large Neural Network that takes 2 minutes for each epoch, and we are training for 10 epochs. We are interested in tuning the learning rate $\alpha \in [10^{-4}, 0.2]$, the L2 regularization strength parameter $\lambda_2 \in [0, 100]$, and the dropout ratio $p \in [0, 0.2]$.

1. If we are doing Grid search with

- $\alpha = 0.0001, 0.0005, 0.001, 0.002, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2$ (10 values)
- $\lambda_2 = 0, 0.1, 1, 5, 10, 100$ (6 values)
- $p = 0, 0.05, 0.1, 0.15, 0.2$ (5 values)

How many epochs in total do we need to train the model to obtain the best performance model among these hyperparameters? How long does it take?

Solution: 300 trials, 3,000 epochs, 6,000 min = 100 hours.

2. Now let's assume that within the domain of α, λ_2, p , there are 5% of the combinations that can result in a really good performance. Then for each trial of the training, we uniformly and independently sample the hyperparameters from their domain. How many trials do we need to run to have a high probability (95%) that the search will find a good performance model? What if we want 99.5% probability to find such a good hyperparameter setting?

Solution: The probability of a fail trial is 95%, and given that

$$\log_{0.95} 0.05 = 58.4 \text{ and } \log_{0.95} 0.005 = 103.29$$

we have

$$1 - (1 - 0.05)^{59} = 0.9515 \text{ and } 1 - (1 - 0.05)^{104} = 0.99518$$

Therefore, we need 59 and 104 trials for 95% and 99.5% confidence, respectively.

Note that this is significantly more efficient than grid search (300 trials) and now we allow not only the discrete value of the hyperparameters. But the cost is that the additional assumption might not always hold and there is still a probability of failure.

Problem 3: Optimization Methods (10 points)

In class we saw several optimization methods. For each of the following techniques explain its advantages over traditional gradient descent (approximately 3 sentences each).

Recall the update rule of Gradient Descent is

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \frac{1}{n} \sum_{i=1}^n \nabla l(\mathbf{w}_t, \mathbf{x}_i)$$

1. (3) Stochastic Gradient Descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla l(\mathbf{w}_t, \mathbf{x}_i)$$

The parameters update more frequently leading to faster convergence. The noise introduced by small updates can help escape getting trapped in narrow local minima. Allows for parallelization. Creates regularization effect.

2. (3) Minibatch Stochastic Gradient Descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \frac{1}{b} \sum_{i=1}^b \nabla l(\mathbf{w}_t, \mathbf{x}_i)$$

In addition to benefits of SGD, has a more stable convergence than SGD, has fewer update steps leading to greater computational efficiency

3. (4) Stochastic Gradient Descent with Momentum

$$m_{t+1} = \mu m_t - \alpha \nabla l(\mathbf{w}_t; \mathbf{x}_i)$$

Can significantly increase convergence rate by accounting for history of gradients, smoothing out updates and speeding up or slowing down step sizes accordingly.

$$\mathbf{w}_{t+1} = \mathbf{w}_t + m_{t+1}$$

Problem 4: Convolutional Neural Networks (25 points)

1. (4) Suppose you have a convolutional filter of size 3×3 . If you apply this filter to a grayscale image of size 36×36 with no padding and stride = 1, what is the dimension of the output? Show your work.

The dimension of the output is 34×34 . The heights and widths of the image would be reduced by 2 to account for hitting the edges, since we have a 3×3 filter, stride = 1, and no padding.

2. (4) Now suppose you apply *two* filters of size $3 \times 3 \times 3$ on an RGB image of size 36×36 with padding = 3 and stride = 2, what is the dimension of the output? Show your work.

Solution: The dimension of the output is $20 \times 20 \times 2$.

The width of the output would be

$$\text{floor}\left(\frac{\text{Input width} + 2 * \text{padding} - \text{filter width}}{\text{stride}}\right) + 1 = \text{floor}\left(\frac{36 + 2 * 3 - 3}{2}\right) + 1 = 20$$

and, similarly, the height would be

$$\text{floor}\left(\frac{\text{Input width} + 2 * \text{padding} - \text{filter height}}{\text{stride}}\right) + 1 = \text{floor}\left(\frac{36 + 2 * 3 - 3}{2}\right) + 1 = 20$$

Each filter would produce a feature map of size 20×20 , which is stacked to give an output of size $20 \times 20 \times 2$.

3. (9) Derive a general formula for the output dimension for convolutional operations. Suppose you have l filters of size $k \times k \times c$ applied to an input image of size $n \times n \times c$ with stride s and padding p , what is the dimension of the output? Explain the general formula you derive.

$$\text{output dimension} = \left(\frac{n - k + 2p}{s} + 1\right) \times \left(\frac{n - k + 2p}{s} + 1\right) \times l \quad (2)$$

First, after padding the input size is $n + 2p$, and we need to account for the filter hitting the edges of the input image, which would give an output of size $\left(\frac{n - k + 2p}{s} + 1\right) \times \left(\frac{n - k + 2p}{s} + 1\right)$

This is the output size for the outputs of each of the l filters, which are stacked to give a final output with dimension in (1).

4. (8) Suppose you are given an 3×3 matrix from an image I and a 2×2 convolutional kernel K . In the image matrix, each entry represents the grayscale color of a corresponding pixel.

$$I = \begin{bmatrix} 6 & 0 & 1 \\ 3 & 4 & 1 \\ 1 & 7 & 0 \end{bmatrix}$$

and

$$K = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

What is the output of applying the filter to the input with stride 1 without padding?

Our output will be

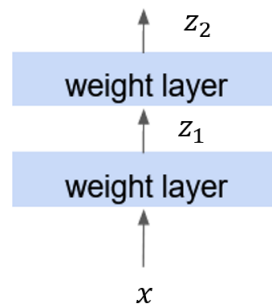
$$\begin{bmatrix} 6 - 4 & 0 - 1 \\ 3 - 7 & 4 - 0 \end{bmatrix} = \begin{bmatrix} 2 & -1 \\ -4 & 4 \end{bmatrix}$$

Problem 5: Residual Networks (25 points)

- (5) What is the vanishing gradient problem in deep neural networks?

The vanishing gradient problem occurs when the gradients used to update the weights become increasingly small. As networks get deeper, more gradients are multiplied together using the chain rule during backpropagation (especially when updating earlier layers). If the gradients are too small, the entire multiplication goes towards 0, which means the weights cannot be updated properly.

- (3) Consider the following (plain) convolutional layers:



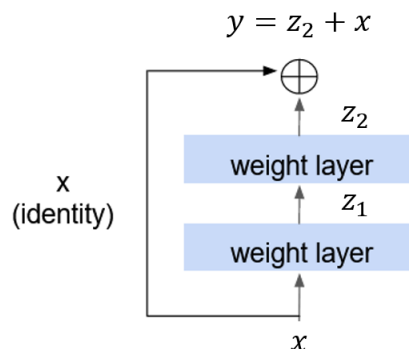
Let x be the input to the convolutions, z_1 and z_2 be the output of the 1st and second layers respectively, and L be some loss function. You are given the following gradients:

$$\begin{aligned}\frac{\partial L}{\partial z_2} &= 0.01 \\ \frac{\partial z_2}{\partial z_1} &= 0.005 \\ \frac{\partial z_1}{\partial x} &= 0.1\end{aligned}$$

What is $\frac{\partial L}{\partial x}$?

Using the chain rule, $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z_2} \times \frac{\partial z_2}{\partial z_1} \times \frac{\partial z_1}{\partial x} = 0.000005$

- (3) Now consider the following residual block:



Once again, let x be the input to the convolutions, z_1 and z_2 be the output of the 1st and second layers respectively, $y = z_2 + x$ be the output of the residual block, and L be some loss function. Assume the gradients are

the same as in part 2(b), and that $\frac{\partial L}{\partial y} = 0.01$. What is $\frac{\partial L}{\partial x}$?

Again using the chain rule, $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \times (1 + \frac{\partial z_2}{\partial x}) = \frac{\partial L}{\partial y} \times (1 + \frac{\partial z_2}{\partial z_1} \times \frac{\partial z_1}{\partial x}) = 0.010005$. As a sanity check, this value should be greater than the value in part 2(b).

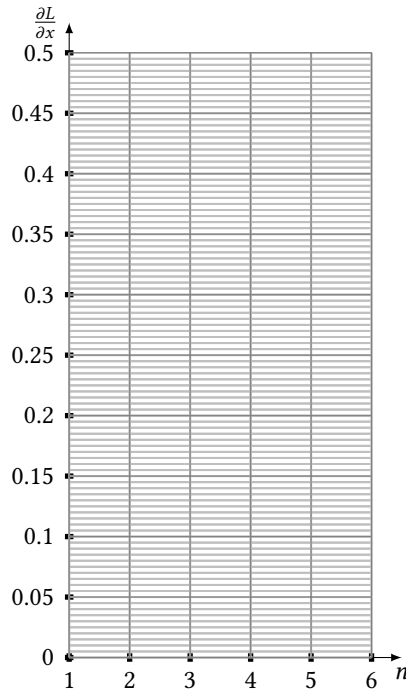
4. (4) Compare the gradients from parts (b) and (c). What do you notice?
5. The gradient from part (b) is much smaller than in part (c), and much closer to 0. This is indicative of the idea of vanishing gradients.
6. (5) Suppose n is the number of convolutional layers in a plain CNN. Let x be the input to the convolutional layers, $z_i \in \{z_1, z_2, \dots, z_n\}$ be the output of the i th convolutional layer, and L be some loss function. You are given the following gradients:

$$\frac{\partial L}{\partial z_n} = 0.5$$

$$\frac{\partial z_i}{\partial z_{i-1}} = 0.5 \quad \forall 1 < i < (n + 1)$$

$$\frac{\partial z_1}{\partial x} = 0.5$$

Graph $\frac{\partial L}{\partial x}$ as the number of layers n increases from 1 to 6.



For layer $n=1$:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial x} = 0.5 * 0.5 = 0.25$$

For layer $n=2$:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z_2} \frac{\partial z_2}{\partial z_1} \frac{\partial z_1}{\partial x} = 0.5 * 0.5 * 0.5 = 0.125$$

For layer n=3:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z_3} \frac{\partial z_3}{\partial z_2} \frac{\partial z_2}{\partial z_1} \frac{\partial z_1}{\partial x} = 0.5 * 0.5 * 0.5 * 0.5 = 0.0625$$

And thus we see the pattern that for each n,

$$\frac{\partial L}{\partial x} = 0.5^{n+1}$$

.

So, our other points are as follows: For layer n=4:

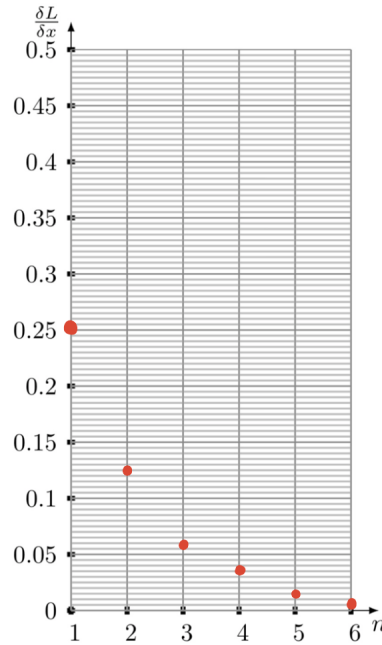
$$\frac{\partial L}{\partial x} = 0.5^{4+1} = 0.5^5 = 0.03125$$

For layer n=5:

$$\frac{\partial L}{\partial x} = 0.5^{5+1} = 0.5^6 = 0.015625$$

For layer n=6:

$$\frac{\partial L}{\partial x} = 0.5^{6+1} = 0.5^7 = 0.0078125$$



7. (5) Now suppose n is the number of convolutional layers in a ResNet, in which a residual block consists of two convolutional layers. Again, let x be the input to the convolutional layers, $z_i \in \{z_1, z_2, \dots, z_n\}$ be the output of the i th convolutional layer, $y_j \in \{y_1, y_2, \dots, y_{n/2}\}$ be the output of the j th residual block, and L be some loss function. You are given the following gradients:

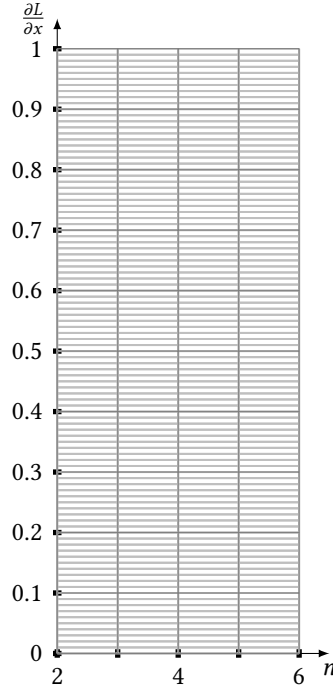
$$\frac{\partial L}{\partial y_{n/2}} = 0.5$$

$$\frac{\partial z_{2i+1}}{\partial y_i} = 0.5 \quad \forall 0 < i < \frac{n}{2}$$

$$\frac{\partial z_{2i}}{\partial z_{2i-1}} = 0.5 \quad \forall 1 \leq i \leq \frac{n}{2}$$

$$\frac{\partial z_1}{\partial x} = 0.5$$

Graph $\frac{\partial L}{\partial x}$ as the number of layers n increases from 2 to 6 (i.e. the number of residual blocks increases from 1 to 3). Note you only need to plot points where n is even.



For layer $n=2$:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y_1} \left(\frac{\partial y_1}{\partial z_2} \frac{\partial z_2}{\partial z_1} \frac{\partial z_1}{\partial x} + \frac{\partial y_1}{\partial x} \right) = 0.5 * (1 * 0.5 * 0.5 + 1) = 0.5(1.25) = 0.625$$

For layer $n=4$:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y_2} \left(\frac{\partial y_2}{\partial y_1} \frac{\partial y_1}{\partial x} + \frac{\partial y_2}{\partial z_4} \frac{\partial z_4}{\partial x} \right) = 0.5 * (1 * 1.25 + \frac{\partial z_4}{\partial z_3} \frac{\partial z_3}{\partial y_1} \frac{\partial y_1}{\partial x}) = 0.5(1.25 + 0.5 * 0.5 * 1.25) = 0.78125$$

Here, we see a pattern that for each n ,

$$\frac{\partial L}{\partial x} = 0.5 * (1.25)^{n/2}$$

.

So, for layer $n=6$:

$$\frac{\partial L}{\partial x} = 0.5 * (1.25)^{6/2} = 0.5 * 1.25^3 = 0.97656$$

.

